UNIVERSITAT POLITÈCNICA DE VALÈNCIA

UPV

Grado en Ciencia de Datos

Infraestructura para el procesamiento de datos

Código de asignatura: GCD-IPD 14016

CURSO 2025-2026

Práctica 02: Utilidades de disco en Linux

Alumnos: Marc Menéndez Zaragozá y Mario Álvarez Martínez

Profesores:

Lenin G. Lemus Zúñiga

Miguel Ángel Mateo Pla

València, 9 de octubre de 2025

Página en blanco.

Contenido

1. Introducción 5
2. Primeros pasos 6
3. Aplicación gráfica *kinfocenter* 7
4. Uso básico de *man* 8
5. Orden ***lsblk*** 10
6. Orden hdparm 13
7. Orden smartctl 15
8. Medida de prestaciones con *dd* 17
8.1. Medir velocidad de escritura con *dd* 18
8.2. Medir velocidad de lectura con *dd* 19

Página en blanco.

1. Introducción

El sistema operativo LINUX es uno de los más usados en los Centros de Proceso de Datos (CPD) de todo el mundo. Aunque LINUX no es UNIX sí que coge de este sistema gran parte de su filosofía de diseño y funcionamiento de aplicaciones. Algo clave en este sistema es todo lo que se puede hacer desde un terminal (físico o remoto) sin capacidades gráficas, con lo que ello implica para la gestión automatizada de los CPD.

En esta práctica nos vamos a utilizar una aplicación gráfica y vamos a ejecutar, en una aplicación de consola que contiene un intérprete de órdenes, las ordenes *lsblk, hdparm y smartctl*:

1.1. **kinfocenter**: aplicación gráfica del entorno gráfico KDE™ que permite obtener información de diferentes componentes, tanto software como hardware del sistema. También sirve como lanzadora/integradora de otras aplicaciones con el mismo objetivo.

1.2. **konsole**: aplicación de consola que contiene un intérprete de órdenes, en esta aplicación se escriben las ordenes a ejecutar.

1.3. **lsblk**: orden que se ejecuta en in interprete de órdenes y que sirve para listar (ls) información sobre los dispositivos orientados a bloques (blk) presentes en el sistema. En el mundo UNIX tienen esta denominación todos los dispositivos usados para almacenar ficheros.

1.4. **hdparm**: orden que se ejecuta en in interprete de órdenes y que permite leer y modificar los parámetros de funcionamiento de dispositivos tipo disco duro (*hard disk* en inglés). También permite hacer algunas pruebas de prestaciones en los discos.

1.5. **smartctl**: orden que se ejecuta en in interprete de órdenes y que utiliza el protocolo SMART presente en algunos discos (tanto HDD como SSD). Este protocolo se utiliza para acceder a parámetros internos de funcionamiento (como la temperatura) que pueden dar pistas sobre la vida útil de los discos, es decir, que permiten detectar si un disco está “normal” o puede empezar a fallar.

El principal objetivo de esta práctica es conocer qué información podemos obtener de estas aplicaciones. Esta información sería de gran ayuda para mantener y ajustar el sistema de almacenamiento de un servidor.

Para la realización de esta práctica necesitamos un ordenador con sistema operativo LINUX nativo, no vale una máquina virtual, en el que tengamos permisos de administrador para las aplicaciones que vamos a usar. Esta memoria y sus ejemplos se han probado en el Laboratorio de Arquitectura de Computadores del DISCA.

Además, para ejecutar algunas de estas órdenes es necesario disponer de permisos de superusuario. En las máquinas del laboratorio se ha concedido a los alumnos de IPD el permiso para ejecutar dichas ordenes como superusuarios. Para ejecutar las ordenes con esos permisos hay que ejecutarlas desde la línea de órdenes anteponiendo la palabra “**sudo**” a la línea de órdenes a ejecutar.

2. Primeros pasos

Al llegar al laboratorio podemos encontrarnos los ordenadores en tres estados:

1. Encendidos con el sistema WINDOWS en marcha. En este caso, apagadlos y seguir al punto 3 de esta lista.
2. Encendidos con el sistema Ubuntu para redes en marcha: apagadlos y seguir al punto 3 de esta lista
3. Apagados: encenderlos y escoger la entrada “KUbuntu” de la lista de posibilidades de arranque.

Una vez identificados en el sistema KUbuntu se ejecutará el entorno de ventanas KDE-Plasma, que es uno de los muchos que hay disponibles en LINUX. Tened en cuenta que la primera vez que se hace esto en una máquina, el sistema tardará unos minutos para crear el entorno.

Una vez identificados tendremos en la pantalla un entorno como el de la Figura 1. Es un entorno muy similar al de Windows, con un menú lanzador de aplicaciones situado en la esquina inferior izquierda, un detalle de este se puede ver en la Figura 2.

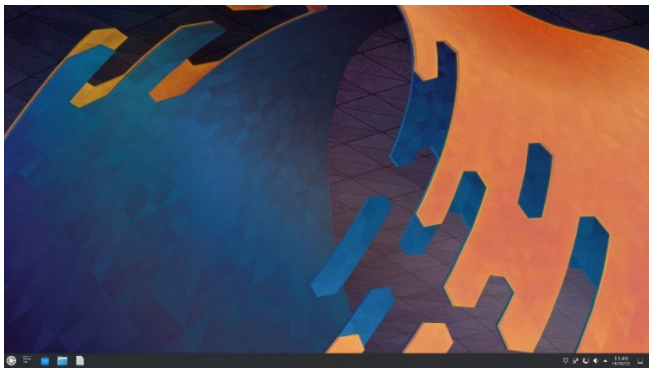


Figura 1. KDE Plasma desktop

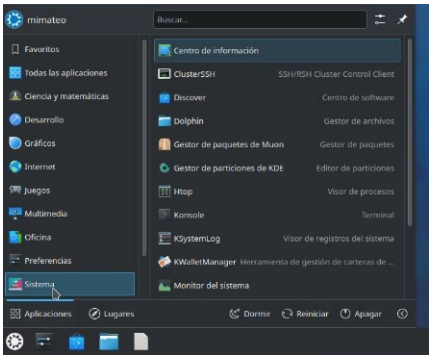
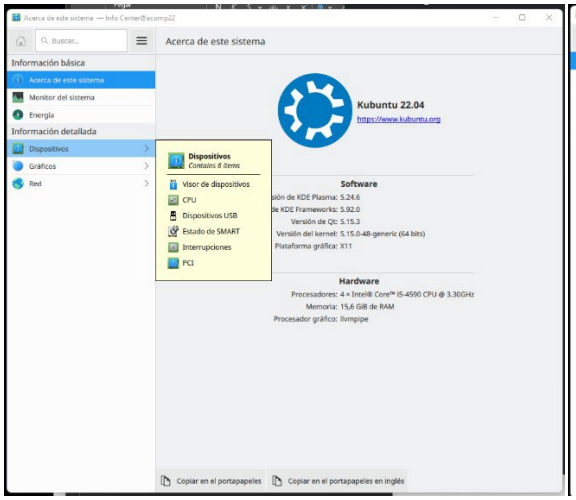


Figura 2. Menú lanzador de aplicaciones

Para la realización de la práctica recomendamos que, en el menú de la Figura 2, abráis una vez la aplicación “Centro de Información” y 2 veces la aplicación de consola con el intérprete de órdenes “*konsole*”, para un total de 3 ventanas en el escritorio, una de ellas para ejecutar las órdenes de la práctica y la otra para acceder al manual en línea (orden *man*). Además, para leer la memoria de la práctica, se puede abrir un navegador web (Chrome y Firefox están instalados) conectado a PoliformaT.

3. Aplicación gráfica *kinfocenter*

Cuando hemos ejecutado la aplicación “Centro de Información” hemos lanzado un programa que en el disco recibe el nombre de *kinfocenter*, esta acción también se puede realizar desde la línea de órdenes (*konsole*).



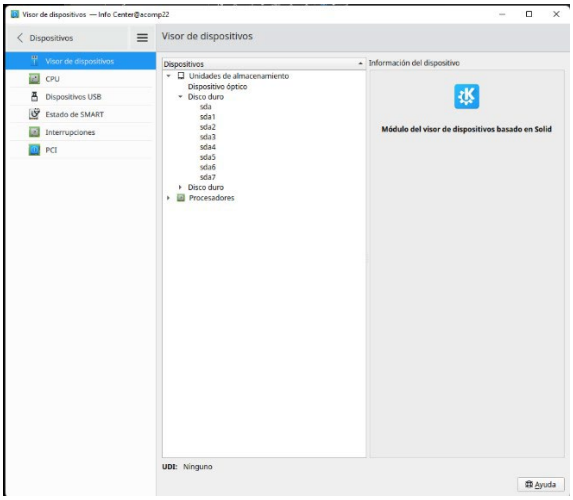


Figura 3. Aspecto inicial kinfocenter Figura 4. Información básica dispositivos

El aspecto inicial de la aplicación es el mostrado en la Figura 3. Por defecto muestra una información muy básica sobre el equipo (versiones del software, tipo de procesador y cantidad

de memoria RAM), pero se pueden escoger vistas alternativas que ofrecerán una información más detalladas sobre otros elementos del sistema.

Esta práctica se centra en la información que se puede obtener de los discos en el sistema, por lo que nos interesará la entrada “Dispositivos”. Si elegimos esa opción se nos mostrará una pantalla como la de la Figura 4.

Esta utilidad es útil si tenemos salida gráfica, nosotros estamos más interesados en aquellos programas que ofrecen información en formato texto que podamos analizar de forma automática.

PREGUNTA 1a. ¿Cuántos discos hay en el sistema en el que estás trabajando según la información mostrada en la aplicación gráfica kinfocenter?

Según el kinfocenter, en el apartado de dispositivos de bloques, hay 3 discos.

PREGUNTA 1b. ¿Qué identificadores reciben?

Son sda, sdb y nvme0n1, los tres de 465,8 GiB. También salen sda1, sda2, sdb1 y nvme0n1p1, p2, p3 y p5, pero eso son particiones de esos discos, no discos aparte.

4. Uso básico de *man*

Para las siguientes secciones trabajaremos siempre desde la línea de órdenes y usaremos mucho el manual en línea. En los sistemas UNIX esto se suele realizar con un programa llamado *man* que tiene diferentes formas de mostrar la información, la más habitual es una salida en un terminal de texto.

En la Figura 5 se muestra la salida de la ejecución de la línea "*man man*" y los campos más importantes de esta salida. En este caso, el programa *man* está usando un segundo programa (*less*) para mostrar la información y recibir indicaciones del teclado, por ejemplo, al pulsar espacio se desplaza la visualización hasta la siguiente página no mostrada o al pulsar la tecla "*q*" se sale de la aplicación. Una tecla interesante es "h", que muestra texto de ayuda para usar "*less*" (de esta ayuda se puede salir pulsando "q").

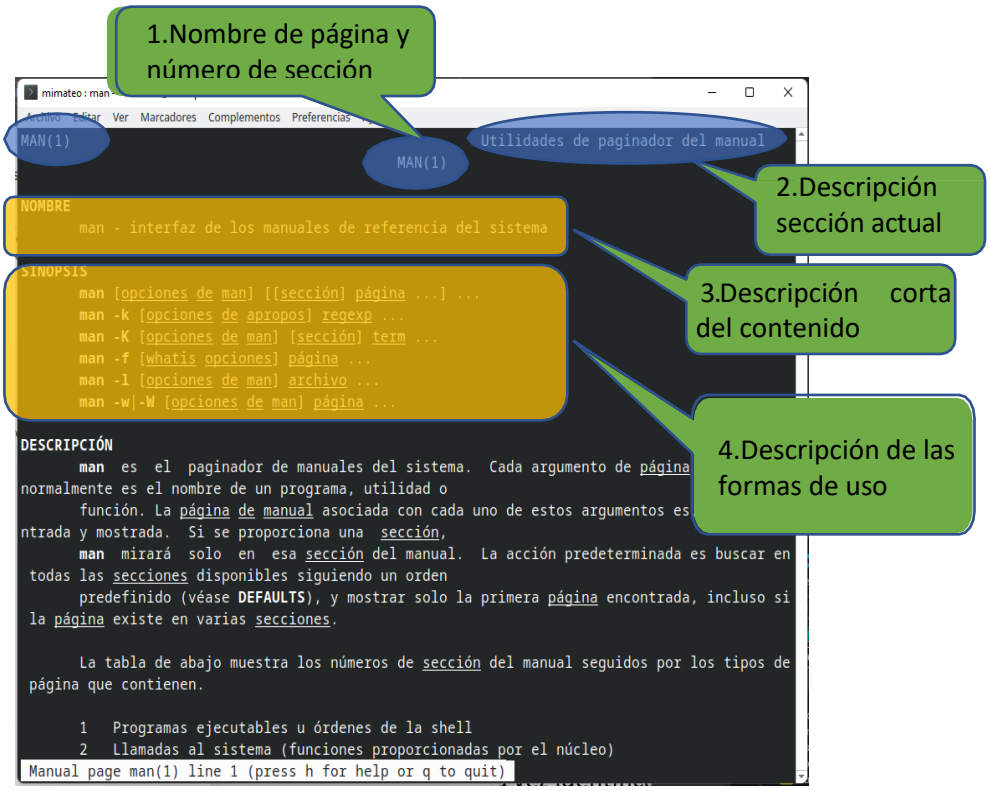


Figura 5. Formato de salida de la orden man

Volviendo a la Figura 5, los campos señalados son muy importantes:

1. Arriba a la izquierda y al principio en el medio SIEMPRE aparece el nombre de la página de manual y, entre paréntesis, la sección del manual en la que nos encontramos. El manual está dividido de forma que puede haber páginas con el mismo nombre en secciones diferentes, pero mostrando información de elementos del sistema diferentes pero que tienen el mismo nombre. Por ejemplo, la página que describe el programa *passwd* y la página que describe el contenido del fichero de configuración *passwd*.
2. Para que quede más claro, siempre hay una descripción de la sección que se está mostrando.
3. La primera parte de una página del manual (primera línea) es una descripción corta de la página actual, donde se deja claro el nombre y la sección en la que se encuentra. El resto de las partes de la página en curso depende de la sección.
4. A continuación, viene un resumen de cómo se utiliza el elemento descrito. En las secciones 1 y 8 es la forma en que se puede lanzar el programa, mientras que, por ejemplo, en las secciones 2 y 3 contiene las líneas de código C para utilizar esa función.

Tras estos apartados viene la descripción detallada del elemento descrito, muchas veces con ejemplos. Es la parte que tendremos que leer, completa o parcialmente, para poder usar el elemento descrito en la página de manual.

Hacia el final de la página de manual aparece otra sección importante: “Mirar También”. En esta sección aparece una lista de páginas de manual (nombre y sección) que están relacionadas con la página de manual mostrada.

PREGUNTA 2. Completar la tabla siguiente		
Nombre	Sección	Número de opciones
man	1	40
lsblk	8	40
hdparm	8	77 (67 + 10 de seguridad)
smartctl	8	32
dd	1	2 (+13 operandos)

5. Orden *lsblk*

En todos los ordenadores tipo UNIX, como LINUX, existe el directorio especial "*/dev*", que no contiene archivos normales. Las entradas de este directorio se utilizan para acceder a los dispositivos (*devices* en inglés) de una forma uniforme en todos los sistemas. De esta forma se evita que en cada sistema haya que aprender una forma de enumerar los dispositivos.

El directorio */dev*, por lo demás, es como cualquier otro directorio, por lo que podemos ver su contenido con los programas de navegación de archivos gráficos o con el uso de *cd* o *ls*. Un ejemplo del uso de *ls* sobre el directorio */dev* en el ordenador acomp22 del laboratorio se puede ver en la Figura 6.

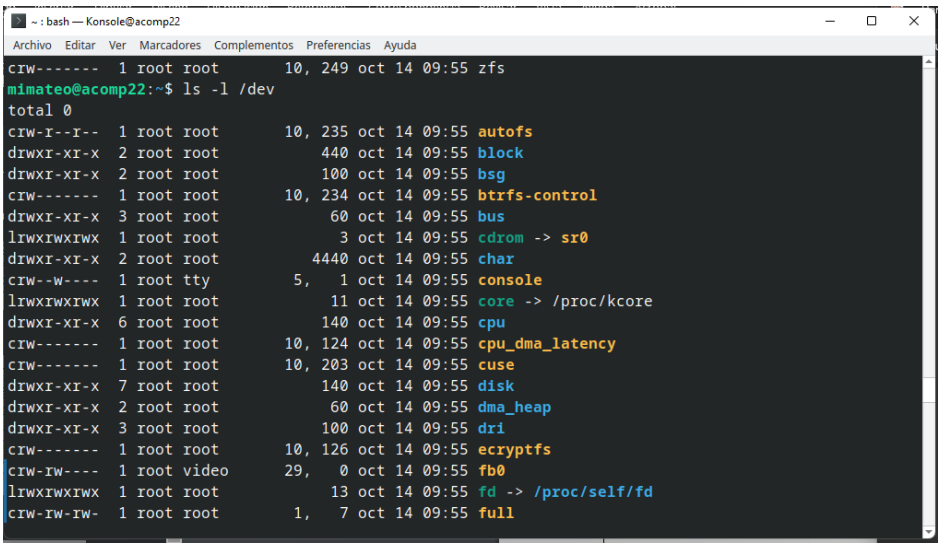


Figura 6. Ejemplo de entradas en */dev* en acomp22

En la salida de la orden *ls*, la primera letra de cada entrada indica el tipo de entrada. En el directorio */dev* es donde vamos a encontrar entradas que empiezan por c (*character*) o por b (*block*). Estas entradas son "dispositivos". En el caso de esta práctica nos interesan los dispositivos orientados a bloque. Los discos duros SATA, los más habituales en los PC, tienen nombres del tipo *sd*[a-z], es decir, "*sd*" y luego una letra. También pueden existir entradas correspondientes a discos SSD conectados con nvme (que empezarían justo por esas cuatro letras)

PREGUNTA 3 En el directorio */dev* del PC del laboratorio en el que estáis,

¿cuántas entradas en /dev se corresponderían con un dispositivo SATA según la regla anterior (nombre sd[a-z]) y cuantos son SSD tipo nvme? (Pon el número de ellos y el nombre de cada uno)

Con la regla sd[a-z] salen 2 entradas de discos SATA: /dev/sda y /dev/sdb, y las dos son de bloques (con ls -l empiezan por «b»). De nvme salen 6 entradas, pero en realidad es un solo SSD: /dev/nvme0n1 es el disco (de bloques), /dev/nvme0 es el controlador (de caracteres, «c») y nvme0n1p1, nvme0n1p2, nvme0n1p3 y nvme0n1p5 son sus particiones.
O sea, 2 discos SATA (sda y sdb) y 1 SSD NVMe (nvme0n1).

CUIDADO: El hecho de que exista una entrada en /dev no implica que exista un dispositivo con ese nombre, sólo indica que el sistema operativo está preparado para manejarlo si existiera.

Para saber qué dispositivos de bloques hay instalados realmente en el sistema y acceder a otras informaciones importantes sobre los mismos, existen mejores herramientas que mirar este directorio, siendo *lsblk* la más extendida.

PREGUNTA 4. Lee la página de manual de lsblk y ejecútala sin parámetros. ¿Cuántos dispositivos diferentes muestras?

Sin parámetros, lsblk nos saca 10 dispositivos de bloques. Si los cuentas por tipo con lsblk -n -o TYPE | sort | uniq -c, son 3 discos (sda, sdb y nvme0n1, de 465,8 GiB cada uno) y 7 particiones (sda1, sda2, sdb1, nvme0n1p1, nvme0n1p2, nvme0n1p3 y nvme0n1p5).

PREGUNTA 5. ¿Qué información puedes extraer de /dev/sda, /dev/sda1 y /dev/sdb y /dev/sb1? Si hay algún otro disco, trata de sacar información también sobre él.

Lo sacamos con lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS,ROTA,TRAN,MODEL y con lsblk -f.
- /dev/sda: es un SSD SATA (ROTA=0, TRAN=sata), un WDC WDS500G2B0A-00SM50 de 465,8 GiB, con 2 particiones.
- /dev/sda1: 200 GiB en NTFS (etiqueta DatosW) y no está montada.
- /dev/sda2: 265,8 GiB en ext4 (etiqueta DatosL), montada en /home, /recursos y /mnt/datos, con un 9 % ocupado.
- /dev/sdb: este sí es un disco duro de los de toda la vida (ROTA=1, TRAN=sata), un Seagate ST500DM002-1BD142, también de 465,8 GiB. Solo tiene una partición de 100 GiB y el resto está sin particionar.
- /dev/sdb1 (en el enunciado pone /dev/sb1, pero suponemos que es una errata): 100 GiB en ext4 (etiqueta HDD_IPD), montada en /mnt/hdd_ipd y vacía (0 %).

Y aparte está el NVMe (Kingston SNV2S500G, 465,8 GiB) con 4 particiones: la EFI de 500 MiB en FAT32 (/boot/efi), 200 GiB en NTFS (Win11), 50 GiB en ext4 montada en / (30 % usado) y 150 GiB en ext4 sin montar.

Entre las diferentes opciones de esta orden, la otra que probaremos será la ”-t”. Esta opción nos mostrará la información sobre la estructura interna del dispositivo.

PREGUNTA 6 Comenta si existen diferencias entre las columnas PHY-SEC y LOGSEC en la ejecución de la orden lsblk -t para los diferentes discos.

PHY-SEC es el tamaño del sector físico, o sea, el que usa el disco de verdad por dentro, y LOG-SEC es el del sector lógico, que es el que ve el sistema operativo.
En sda (SSD SATA) y en nvme0n1, y en sus particiones, los dos son de 512 B, así que ahí no hay ninguna diferencia. Donde sí cambia es en sdb (el disco duro) y en sdb1: 4096 B de sector físico y 512 B de lógico. Básicamente es un disco «512e»: por dentro guarda los datos en sectores de 4 KiB, pero hacia fuera se presenta con sectores de 512 B para ser compatible. Por eso también tiene MIN-IO = 4096, porque lo que mejor le va es leer y escribir en múltiplos de 4 KiB.

6. Orden hdparm

CUIDADO: Con esta orden nos metemos en terreno peligroso. La orden puede usarse para obligar a los dispositivos a trabajar con opciones potencialmente destructivas para ellos o para la información que contienen. Antes de ejecutar cualquier ***hdparm*** sobre un dispositivo, aseguraos bien que las opciones son correctas y el dispositivo soporta la acción que queréis hacer. Para ello es IMPRESCINDIBLE, leer bien la página de manual y la información del fabricante del dispositivo.

ATENCIÓN: Esta orden NO se puede ejecutar sin permisos de superusuario, anteponer siempre "sudo" antes de ejecutarla.

PREGUNTA 7.a Describe la información que aparece al ejecutar la siguiente línea:

sudo hdparm /dev/sda

Este es el SSD SATA (el WDC de 500 GB). Sin opciones, hdparm te saca esto:
- multcount = 0 (off): no usa lo de transferir varios sectores por orden (en discos SATA que van por DMA no se usa).
- IO_support = 1 (32-bit): la E/S entre el sistema y la controladora va a 32 bits.
- readonly = 0 (off): no está protegido contra escritura, se puede leer y escribir.
- readahead = 256 (on): lee 256 sectores por adelantado (128 KiB), que es lo mismo que el RA = 128 de lsblk -t.
- geometry = 60801/255/63, sectors = 976773168, start = 0: la geometría C/H/S es inventada, está solo por compatibilidad. Lo que importa es el número de sectores: 976 773 168 × 512 B ≈ 500 GB (465,8 GiB). Y start = 0 porque es el disco entero, no una partición.

PREGUNTA 7.b Describe la información que aparece al ejecutar la siguiente línea:

sudo hdparm /dev/sdb

Este es el disco duro (Seagate ST500DM002) y sale casi lo mismo que en sda:
- multcount = 16 (on): aquí sí está activado, mueve hasta 16 sectores en cada orden de lectura o escritura.
- IO_support = 1 (32-bit), readonly = 0 (off) y readahead = 256 (on), igual que en el SSD.
- geometry = 60801/255/63, sectors = 976773168, start = 0: misma geometría y mismos sectores, porque los dos discos son de la misma capacidad (unos 500 GB, 465,8 GiB).

Vamos, que la única diferencia real es el multcount: el disco duro mueve 16 sectores por orden y en el SSD eso está apagado.

PREGUNTA 8 Compara y explica las diferencias de la información recogida con las opciones "-i" y "-I" sobre uno de los dispositivos de bloques de la máquina en la que estás.

Lo probamos en sda (el SSD WDC). Con -i, hdparm te enseña lo que el kernel leyó del disco al arrancar y tiene guardado. Es un resumen corto y con un formato bastante viejo, de cuando los IDE: modelo, firmware (401000WD), número de serie, una geometría CHS heredada (16383/16/63), los 976 773 168 sectores LBA, los modos PIO, DMA y UDMA que soporta (el activo va con *, udma6) y que tiene la caché de escritura activada. Hay varios campos que salen como unknown o 0 porque ese formato no los recoge.

Con -I, en cambio, hdparm le pregunta directamente al disco (con la orden ATA IDENTIFY DEVICE) y te lo da todo mucho más completo: SATA 3.0 a 6 Gb/s, los estándares ATA que soporta, sector lógico y físico de 512 B, la capacidad exacta (500 GB), que es de 2,5", que la velocidad de giro es «Solid State Device» (o sea, que es un SSD), cola NCQ de 32 y la lista de funciones con un * en las que están activas (SMART, caché de escritura, NCQ, TRIM...). También salen el estado de seguridad, el WWN y el checksum.

Resumiendo: -i es lo que se acuerda el kernel del disco y -I es la información completa que te da el propio disco.

PREGUNTA 9 Escribe los resultados de la ejecución de hdparm con las opciones “-t” y “-T” sobre cada uno de los dispositivos de bloques de la máquina en la que estás. Explica las diferencias obtenidas en el mismo dispositivo y entre dispositivos (indicando para cada uno si es HDD o SSD)

-T (cached reads) lee de la caché en RAM sin tocar el disco, así que en realidad lo que mide es la memoria. -t (buffered disk reads) sí que lee del disco de verdad. Nos salió esto (media de 3 veces):
- sda (SSD SATA WDC): -T ≈ 4806 MB/s, -t ≈ 1,98 MB/s
- sdb (HDD Seagate): -T ≈ 22325 MB/s, -t ≈ 133,4 MB/s
- nvme0n1 (SSD NVMe Kingston): -T ≈ 22995 MB/s, -t ≈ 1280 MB/s

En el mismo disco, -T es muchísimo más alto que -t porque leer de RAM es muchísimo más rápido que leer del disco. Entre discos, -T sale parecido en sdb y en nvme0n1 (la memoria es la misma), y con -t ya se ve la diferencia de verdad: el NVMe va como 10 veces más rápido que el HDD, y el HDD da unos 133 MB/s, que cuadra con su hoja de características (125 MB/s de media y 144 MB/s de máximo). Lo raro fue el SSD SATA: nos dio unos 2 MB/s cuando tendría que andar por los 500 MB/s, y el -T también salió más bajo y variando. Lo repetimos un rato después y volvió a dar unos 2,6 MB/s. Pensamos que el disco o el sistema estaban ocupados en ese momento (en ese SSD está /home), porque otro día, con dd, el SSD SATA leyó a 475 MB/s, que ya es lo normal.

PREGUNTA 10 ¿Por qué sería necesario ejecutar varias veces el test anterior?

Porque con una sola medida no te puedes fiar. Mientras hdparm está midiendo, el ordenador sigue haciendo cosas: el sistema y otros programas pueden estar usando el disco, la CPU o la memoria, y eso te baja el resultado. Además hay cosas que cambian de una vez a otra, como lo que ya hay en la caché o a qué frecuencia va el procesador en ese momento. El propio man de hdparm dice que repitas la prueba 2 o 3 veces con el sistema sin hacer nada más. Así puedes hacer la media y ver si alguna ha salido rara. De hecho, en la pregunta 9 los tres valores de cada disco no nos salieron iguales.

Recuerda que un fichero de 1203 B en un dispositivo de bloques en el que cada bloque o sector es de 512 B, ocupa al menos 3 sectores, dependiendo de cómo el sistema de ficheros reparta los “sectores” (bloques en el disco) en “bloques” (trozos de sectores desde el punto del programa que implementa el sistema de ficheros).

PREGUNTA 11 Describe un ejemplo de uso del parámetro “–fibmap” y para que podría servir.

--fibmap te dice en qué sectores del disco está guardado un fichero. Por ejemplo, creamos un fichero de 1203 bytes con head -c 1203 /dev/urandom > ~/f1203 y luego hicimos sudo hdparm --fibmap ~/f1203. Nos dijo que el sistema de ficheros usa bloques de 4096 B, que la partición (sda2) empieza en el sector LBA 419430463 y que el fichero está en un solo trozo, del sector 660603887 al 660603894, o sea, 8 sectores de 512 B. Con 3 sectores le valdría para 1203 B, pero ocupa 8 porque el sistema de ficheros reserva bloques enteros de 4096 B (8 × 512 B).
Sirve por ejemplo para ver si un fichero está fragmentado (si salen varias filas es que está repartido en varios trozos), para saber qué sectores exactos tienes que leer si quieres recuperar datos de un disco que está dañado, o para ver si un sector defectuoso le afecta a un fichero en concreto.
Primero lo probamos con un fichero en /tmp y nos dio el error «0,36: device not found in /dev». Es porque en estos PC /tmp es un tmpfs que está en la RAM, no en un disco.

7. Orden smartctl

CUIDADO. Como en el caso de la orden hdparm, algunas opciones de esta orden pueden destruir la información o el disco sobre el que se opera.

La orden *smartctl* permite acceder al sistema S.M.A.R.T. si este está presente en el disco y el método de conexión es compatible con el mismo. Inicialmente sólo era compatible con dispositivos IDE conectados directamente al ordenador. Hoy en día está presente en casi todos los dispositivos de bloques de tipo HDD y SSD conectados con SATA o SAS. Incluso muchos discos conectados por USB permiten acceder a este tipo de información.

Este protocolo y conjunto de informaciones están pensados para determinar la "salud" del disco, y hacerlo con acciones realizadas desde el propio disco con poca o nula intervención del host en el que el disco está conectado. Por desgracia, parte de la información devuelta depende del fabricante, en esos casos no queda más remedio que acudir a la documentación del fabricante para interpretar correctamente la información.

Eso sí, la información se recupera siempre del dispositivo y es este el encargado de generarla y realizar un análisis preliminar de la misma.

Si te fijas en la Figura 4 puedes ver que **kinfocenter** incluye información de esta orden. En concreto muestra en un cuadro de texto la salida de la línea "smartctl -a". Lo mejor es que sigáis el artículo de Bruce Allen para *Linux Journal* titulado "Monitoring Hard disks with SMART", que podéis encontrar en:

https://www.linuxjournal.com/magazine/monitoring-hard-disks-smart

Antes de ejecutar las órdenes que se sugieren en dicho documento, leed primero las cuestiones que tenéis que contestar. Tened en cuenta que queremos sólo lo que tiene que ver con *smartctl* y no con *smartd*. También serán útiles las siguientes páginas:

https://www.thomas-krenn.com/en/wiki/Category:Smartmontools

https://www.techrepublic.com/blog/linux-and-open-source/using-smartctl-to-get-smart-status-i

https://milmazz.uno/article/2006/09/25/smartmontools-aprendiendo-a-chequear-tu-disco-duro/

https://codeyarns.com/2016/12/21/how-to-use-smartctl/

PREGUNTA 13 ¿Cuál es el estado de cada uno de los discos del sistema tal y como están al principio?

Antes de lanzar ningún test, los tres discos pasan la autoevaluación de SMART: sudo smartctl -H nos dio «SMART overall-health self-assessment test result: PASSED» en sda (SSD SATA), en sdb (HDD) y en nvme0n1 (SSD NVMe). O sea, que según los propios discos ningún atributo está por debajo de su umbral de fallo.
Eso sí, en sda choca un poco con lo lento que salió con hdparm -t. Que ponga PASSED solo quiere decir que ningún atributo ha llegado al umbral crítico, no que el disco esté funcionando al cien por cien.

PREGUNTA 14 ¿Cuál ha sido la temperatura máxima que ha tenido cada disco?

Lo miramos con sudo smartctl -A y, en los que se podía, también con sudo smartctl -l scttemp.
- sda (SSD SATA): el atributo 194 marca 28 °C ahora y un mín./máx. de 13/51 °C, así que la máxima ha sido 51 °C. Este disco no tiene historial SCT.
- sdb (HDD): está a 31 °C. En este encendido ha llegado como mucho a 31 °C y en toda su vida a 50 °C (Lifetime Min/Max), que está dentro de lo que dice el fabricante (14-55 °C).
- nvme0n1: está a 30 °C y sus sensores marcan 57, 33 y 34 °C. No guarda la máxima de toda su vida, pero los contadores Warning y Critical Comp. Temperature Time están a 0, así que nunca ha pasado de la temperatura de aviso.

PREGUNTA 15 ¿Cuáles han sido los tiempos de funcionamiento de los discos?

Con sudo smartctl -A miramos el atributo 9 (Power_On_Hours) en los SATA y el campo Power On Hours en el NVMe:
- sda (SSD SATA): 2218 horas encendido (unos 92 días, unos 3 meses).
- sdb (HDD): 11240 horas (unos 468 días, casi 1 año y 3 meses).
- nvme0n1 (SSD NVMe): 1310 horas (unos 55 días) y 938 encendidos.

El disco duro es el que más lleva funcionando con diferencia: unas 5 veces más que el SSD SATA y más de 8 veces más que el NVMe. En el NVMe, 938 encendidos para 1310 horas sale a menos de hora y media cada vez, que es lo normal en un PC de laboratorio que se enciende y se apaga en cada clase.

PREGUNTA16 Tras realizar un short test. ¿Cuánto ha tardado en realizarlo y en qué estado ha quedado cada disco?

Lanzamos el test corto (sudo smartctl -t short) en los tres discos a las 12:32:08. Según el propio disco, tardaría 2 minutos en sda (SSD SATA), hasta las 12:34:42, y 1 minuto en sdb (HDD), hasta las 12:33:47. Cuando miramos el registro con sudo smartctl -l selftest a las 12:39:47 ya habían acabado los dos: la última entrada (#1) pone «Short offline - Completed without error», con un 00 % restante y sin ningún LBA con error (a las 2218 h de vida en sda y a las 11240 h en sdb). Así que los dos están bien. En el registro también salen tests cortos de antes, todos sin errores (13 en sda y 9 en sdb).

El NVMe (nvme0n1) no admite el autotest SMART: tanto al lanzarlo como al mirar el registro te contesta «Read Self-test Log failed: Invalid Field in Command (0x002)», así que en ese disco no se pudo hacer.

8. Medida de prestaciones con *dd*

Una forma básica de medir prestaciones de un disco es realizar operaciones de lectura y escritura en el mismo y medir el tiempo que se tarda en realizar estas acciones. Debido a la existencia de las caches de E/S esta tarea puede dar resultados mucho mayores de los reales del disco.

Una orden que puede ser útil para realizar esta tarea de medida de prestaciones es la orden ***dd***. El nombre proviene de su utilización original: "disk duplication". Esta orden lee datos de un fichero de origen y escribe esos datos en un fichero destino.

La utilización básica que nosotros realizaremos será del siguiente tipo:

~:> dd if=**ruta1** iflag=**opts_in** of=**ruta2** oflag=**opts_out** bs=**X** count=**Y** status=**level**

Esta orden leería desde **ruta1** un total de **Y** bloques, cada uno de **X** bytes, escribiéndolos en **ruta2**. Para las lecturas se utilizan las opciones **opts_in** y para las escrituras las **opts_out**. Durante la ejecución, y al finalizar la misma, el nivel de información es definido por lo pasado como **level** a la opción status**.**

Como esta orden realiza tanto lecturas como escrituras es necesario utilizar unas rutas especiales para ruta1 y ruta2 si queremos medir sólo tiempos relacionados con lectura o con escritura.

PREGUNTA 17 ¿qué diferencia hay entre /dev/null y /dev/zero (**man null**) si se utilizan para leer de ellas? ¿y si se utilizan para escribir?

Según man null:
- Al leer: /dev/null no te da nada, la lectura acaba al momento con fin de fichero (0 bytes). /dev/zero, en cambio, te da todos los bytes a cero ('\0') que le pidas y no se acaba nunca.
- Al escribir: los dos hacen lo mismo. Se tragan todo lo que les escribas, la escritura siempre sale bien, pero los datos se tiran.

Por eso en la práctica se usa /dev/zero como origen (if=) en las pruebas de escritura, para tener datos sin tener que leerlos de ningún disco, y /dev/null como destino (of=) en las de lectura, para que lo que se lee no se escriba en ningún sitio y solo se mida la lectura.

Con la opción bs (o las opciones ibs y obs) permiten controlar los procesos de lectura y escritura, realizando operaciones más o menos grandes. Este extremo es importante como veremos en las siguientes pruebas.

Nosotros realizaremos las pruebas sobre los SSD creando un fichero de 1G byte (2^{30} bytes), pero con diferentes tamaños de **bs** expresados como potencias de 2. Para las pruebas realizaremos las siguientes combinaciones:

bs	count	Total
1b	2M	1G
1K	1M	1G
4K	256K	1G
64K	16K	1G

8.1. Medir velocidad de escritura con *dd*

Para realizar las pruebas de escritura el valor de if tiene que ser ***/dev/zero***. Para el fichero de salida necesitamos poder escribir en un directorio del disco a evaluar. Además, para evitar el efecto de la cache de ES, usaremos las siguientes opciones: "**oflag=direct conv=fdatasync**". El nivel de información (**status**) puede no ponerse o utilizar "**progress**". El **iflag** no se utilizará en estas pruebas.

En el laboratorio hay 3 discos y en cada uno de ellos hay al menos un directorio en el que tenemos permisos de escritura.

PREGUNTA 18. Dado que hay 3 discos en el laboratorio completa la siguiente tabla en la que se indica una posible ruta para cada disco:



Directorio	iD_meoria	Unidad	File System
/tmp	ID1	tmpfs (en RAM)	tmpfs
$HOME	ID2	/dev/sdb2 (SSD SATA)	ext4
/mnt/HDD_IPD	ID3	/dev/sda1 (HDD)	ext4

PREGUNTA 19: Para cada uno de los directorios y tamaños (diferentes bs) completar la siguiente tabla con la velocidad de escritura en MB/s:

bs	count	ID1	ID2	ID3
1b	2M (HDD: 256K)	1169 MB/s	4,6 MB/s	2,9 MB/s
1K	1M (HDD: 128K)	2050 MB/s	8,5 MB/s	5,4 MB/s
4K	256K (HDD: 32K)	4624 MB/s	132 MB/s	32,3 MB/s
64K	16K (HDD: 2K)	6246 MB/s	369 MB/s	78,7 MB/s

Orden tipo:

$>dd if=/dev/zero of=***IDx****/fichero_grande oflag=direct bs=****XX*** *count=****YY*** *status=progress conv=fdatasync*

Para el caso de */dev/hdb1*, se debe crear un archivo de 128MBytes. En este caso utilizar en ***count*** los siguientes valores: 256M, 128M, 32M, 2M.

8.2. Medir velocidad de lectura con *dd*

Para realizar las pruebas de escritura el valor de of tiene que ser /dev/null. Para el fichero de entrada usaremos los ficheros creados anteriormente. Además, para evitar el efecto de la cache de ES, usaremos las siguientes opciones: "**iflag=direct,sync**". El nivel de información (**status**) puede no ponerse o utiliza "**progress**". El **oflag** no se utilizará en estas pruebas.

PREGUNTA 20

Para cada uno de los directorios y tamaños (diferentes bs) completar la siguiente tabla con la velocidad de lectura en MB/s:

bs	count	ID1	ID2	ID3
1b	2M (HDD: 256K)	1480 MB/s	27,8 MB/s	4,8 MB/s
1K	1M (HDD: 128K)	2816 MB/s	53,9 MB/s	10,1 MB/s
4K	256K (HDD: 32K)	7252 MB/s	153 MB/s	34,8 MB/s
64K	16K (HDD: 2K)	12703 MB/s	475 MB/s	81,6 MB/s

Orden tipo:

$>dd if=***IDx****/fichero_grande of=/dev/null iflag=direct,sync bs=****XX**** count=****YY**** status=progress*

Recordad que en caso de /dev/hdb1 se han creado archivos más pequeños.

ATENCIÓN: recordad borrar los tres ficheros creados antes de apagar el ordenador.
No hacerlo supondrá una penalización en la nota de prácticas.

Se podría realizar pruebas más exhaustivas con otras órdenes o con programas específicos, pero estas ya dan una idea bastante aproximada de las diferencias entre 3 tipos de discos. Para ver otros ejemplos se pueden consultar, entre otras, las siguientes páginas:

- https://commandmasters.com/commands/fio-common/
- https://fostips.com/kdiskmark-free-hdd-ssd-benchmark-tool-linux/
- https://www.cyberciti.biz/faq/howto-linux-unix-test-disk-performance-with-dd-command/